Informatics Notes

aperson31415

Now just for ORAC

Built 2026-04-23

Changed from AIO to ORAC (AIIO/FARIO) notes.

Contents

1 Helper Snippets 1

2 Strategy 2
- 2.1 Question 2
- 2.2 Competition 2
- 2.3 Debugging 2

3 Topics 3
- 3.1 Greedy Algorithms 3
- 3.2 Brute Force 3
- 3.3 DP 3
- 3.4 Binary Search 3
- 3.5 Monotone Stack 3
- 3.6 Casework 3
- 3.7 Prefix/Suffixes 3
- 3.8 Pointers 3
- 3.9 Graphs 4
- 3.10 Data Structures 4
- 3.11 Other small ideas 4

4 Advanced Topics 5
- 4.1 Shortest Path 5
- 4.2 DSU 5
- 4.3 DP 6
- 4.4 Bashing Techniques 6
- 4.5 Range queries 6
- 4.6 Trees 10
- 4.7 Tree Structures 11
- 4.8 Advanced DP 11
- 4.9 Other graph theory 11

5 Stress Testing 13

6 Miscellaneous 14
- 6.1 Binary Search Variants 14
- 6.2 Minimum-storing data structures 14
- 6.3 Order Statistic Tree 15
- 6.4 Mathematics 16
- 6.5 Bits 17
- 6.6 Other 18
- 6.7 Practice 18
- 6.8 Environment 19
- 6.9 Topic Searching 19

Helper Snippets (1)

boilerplate.cpp

9 lines

```
#pragma GCC optimize("O3")
#include <bits/stdc++.h>
#pragma GCC target("avx,avx2,bmi,bmi2,lzcnt,popcnt,sse4,fma")
using namespace std;

unsigned main() {
    cin.tie(0)->sync_with_stdio(0);
    cin.exceptions(cin.failbit);
}
```

(Most) of these functions/macros are quite short, so you should try practice typing them out, and use them in contest. However, most of them just save some typing time and improve readibility, so feel free to use none of them.

shortmacros.cpp

13 lines

```
#define len(x) ((int) (x).size())
#define elif else if
#define until(x) while(!(x))
#define all(x) (x).begin(),(x).end()
#define  in  :
#define  is  ==
#define  isnt  !=
using ll = long long;
#define _min(x, y) ((x) < (y) ? (x) : (y))
#define _max(x, y) ((x) > (y) ? (x) : (y))

int _lg(int& x) {return 31 - __builtin_clz(x);}
int _lg(ll& x) {return 63 - __builtin_clzll(x);}
```

pythonicrange.cpp

19 lines

```
// Negative step doesnt work yet

struct Range {
    int start, stop, step;

    struct Iterator {
        int value, step;
        int operator*() const {return value;}
        bool operator!=(const Iterator& other) const {return value < other.value;}
        void operator++() {value += step;}
    };

    Iterator begin() const {return {start, step};}
    Iterator end() const {return {stop, step};}
};

inline Range range(int end) { return {0, end, 1}; }
inline Range range(int start, int end) { return {start, end, 1}; }
inline Range range(int start, int end, int step) { return {start, end, step}; }
```

fileio.cpp

2 lines

```
freopen("in.txt", "r", stdin);
freopen("out.txt", "w", stdout);
```

If for some reason you forget or lose the compilation flags, here they are:

compile.sh

5 lines

```
# safe
g++ -std=c++17 sol.cpp -Og -Wall -Wextra -Wshadow -Wconversion -Wfloat-equal -
    Wduplicated-cond -Wlogical-op -Wshift-overflow=2 -Wcast-qual -Wcast-align -
    D_GLIBCXX_DEBUG -D_GLIBCXX_DEBUG_PEDANTIC -D_FORTIFY_SOURCE=2 -fsanitize=
    address -fsanitize=undefined -fno-sanitize-recover -fstack-protector -Wnull-
    dereference -Wextra-semi -Wuninitialized -Winit-self -fstack-clash-protection -
    Wduplicated-branches

# normal judge probably
g++ -std=c++17 sol.cpp -O2 -Wall
```

Also, just a compilation trick, use '-Dlocal' as an additional flag to define the macro 'local'. This is very useful for lots of reasons, such as easy i/o method transferring (some ojs need file i/o).

Strategy (2)

Too short rn

2.1 Question

Unless the problem is super easy, solve every subtask, see how smaller ones match up or can be optimised/joined together/generalised.

Make a buncha observations and do problems by hand, on paper!

Search for techniques and alternate representations, simplify the problem. Maybe see if this problem is related to one you've don in the past.

2.2 Competition

Solve easy problems first, then think on hard ones.

Go to toilet if you really cant think.

In the last half an hour, code some fast solves that may or may not even work (make sure to get EVERY subtask u can).

Use all of the resources you have (cph, cppreference etc).

2.3 Debugging

Check if your logic is even correct. Check if you even read the question correctly!

Check edge cases in your logic and code, and small things like set vs multiset (set vs vector can also be differenc between tle and ac).

Read ALL of your code, maybe even just make it more readable, then inspect each line closely.

Topics (3)

3.1 Greedy Algorithms

Takes best choice at every step, from sorting, from edges, from middle, etc.

Most of the time, you would and should have enough time to prove these with exchange argument.

A greedy algorithm could also commonly just be a part of a bigger problem, in some q5s/6s.

3.2 Brute Force

For q1 and 2 and maybe 3, just brute force it, even if there is a way better solution.

Remember there are lots of different ways to brute force.

3.3 DP

Some are trivial DP, some are innovative, like stack von treap dude.

Remember complexity is $O(\text{base cases} + \text{states} \cdot \text{transition time})$.

Just see how you could make subproblems out of a task. For small constraints, some seemingly-bashy states could even work.

Some optimisations might be useful. More on DP in Ch 4.

3.4 Binary Search

Binary search normally is usually a part of some q4s-6s.

In c++, just use upper and lower bound for normal bsearch.

If monotonic question, very easily could be binary search on ans, if check can be in something like $O(N)$

bsearchans.cpp

6 lines

```
int lo = 0, hi = MAX + 1;
while (lo + 1 < hi) {
    int mid = (lo + hi) / 2;
    if (check(mid)) lo = mid;
    else hi = mid;
}
```

Variants of bsearch in Ch 4.

3.5 Monotone Stack

Yes very good for smaller to left/right (reverse order), also can be bigger to left/right, in amortised $O(N)$

monotonestk.cpp

13 lines

```
template <typename T>
vector<T> closestsmallleft(vector<T> elems) {
    // Array positions, not values
    vector<T> small_toleft(elems.size());
    stack<pair<T, int>> stk;
    stk.push({0, 0});
    for (int i = 1; i <= (int) elems.size(); i++) {
        while (!stk.empty() and stk.top().first >= elems[i - 1]) stk.pop();
        small_toleft[i - 1] = stk.top().second;
        stk.push({elems[i - 1], i});
    }
    return small_toleft;
}
```

3.6 Casework

Just do everything by hand and quadriple check until ur like 'surely this isnt wrong' (for the 2nd time).

Also when implementing, can do by hand first cuz easy to edit, but check very closely.

3.7 Prefix/Suffixes

Prefix and suffix min/max is good, sums and products and xors are good.

2+ dimensioned psums is just inclusion/exclusion principle, but kinda annoying to implement cuz have to get psums of each row first.

Also part of this could be difference arrays, with 2d being even more annoying.

3.8 Pointers

2 pointers is VERY versatile amortised technique, can be used in unexpected ways.

Somehow I'm super bad at implementing 2 pointers, so here:

2ptrs.cpp

15 lines

```
int right = 0, curr;
for (int left = 0; left < n; left++) {
    while (right < n && /* right condition */) {
        // Update state/curr
        right++;
    }

    if (right > left) {
        // Prepare for changing left
    } else {
        // Move right forwards too
        right = left + 1;
        // Update the range
    }
}
```

3.9 Graphs

Most are pretty easy with adj list and some sort of algorithm. More on graphs in next section.

Some times keys will be different (i.e. 2d or strings) but algorithms will work with small adjustments.

DFS in $O(V + E)$ can find connected components well, is good enough for most graph questions, can be used in unexpected ways tho.

dfs.cpp

9 lines

```
template <typename T>
void dfs(T s) {
    if (visited[s]) return;
    visited[s] = true;
    // process node s
    for (T u: adj[s]) {
        dfs(u);
    }
}
```

BFS in $O(V + E)$ is needed for peth length usually, a bit longer. Also 1/0 shortest path. If you can, just use DFS tho.

bfs.cpp

17 lines

```
template <typename T>
void bfs(T x) {
    queue<T> q;
    visited[x] = true;
    dist[x] = 0;
    q.push(x);
    while (!q.empty()) {
        T s = q.front(); q.pop();
        // process node s
        for (T u : adj[s]) {
            if (visited[u]) continue;
            visited[u] = true;
            dist[u] = dist[s]+1;
            q.push(u);
        }
    }
}
```

Flood fill in $O(V + E)$ is good for grids, but u can probably implement with dfs and bfs qucker.

floodfill.cpp

14 lines

```
void floodfill(int r, int c, int group) {
  if ((r < 0 or r >= rows or c < 0 or c >= cols)
      or matrix[r][c] != group
      or visited[r][c]
  ) return;
  visited[r][c] = true;
  curr_size++;

  floodfill(r, c + 1, group);
  floodfill(r, c - 1, group);
  floodfill(r - 1, c, group);
  floodfill(r + 1, c, group);
}
```

3.10 Data Structures

You can use multiple data structures to represent data at the same time to gain multiple types of functionality. Also, intervals -¿ sets most of the time, use interval tricks.

Most of the time, don't worry about memory unless it's small, or you get mle.

Also, when implementing any problem, remember pqs, sets, maps, etc and all of the functionality that they have.

3.11 Other small ideas

Coordinate compression can be very good sometimes, with dp as well. Deques can be used to emulate sliding windows (or with some advanced data structures) Meet in the middle maybe needed.

Interval trick: prefix sums and graph for 'in this interval, this is needed'. Also, otherwise use a set (if no overlaps just use start & end)

Advanced Topics (4)

4.1 Shortest Path

Graph algorithms can be modified parameters etc is very useful.

Remember about bfs btw as well. Also, yet to put here, but multi-source trick is very useful (in maybe aiio/fario, not aio probably).

Bellman-Ford algorithm, shortest path in $O(VE)$ with negatives allowed, and negative cycle detection.

bellmanford.cpp

8 lines

```
dist[x] = 0;

for(int i = 1; i < n; i++){
    for(auto& [a, b, w] : edges){
        // edge with length w goes from a to b
        dist[b] = min(dist[b], dist[a] + w);
    }
}
```

Dijkstra's algorithm, $O((V+E)\log E)$, but negatives are cooked.

dijkstra.cpp

28 lines

```
template <typename T>
void dijkstra(T start_node) {
    priority_queue<pair<int, T>, vector<pair<int, T>>, greater<pair<int, T>>> q;

    vector<int> dist(N + 1, INT_MAX);
    vector<bool> processed(N + 1, false);

    dist[start_node] = 0;
    q.push({0, start_node}); // {dist, node}

    while(!q.empty()) {
        T a = q.top().second;
        q.pop();

        if(processed[a]) continue;
        processed[a] = true;

        for(auto u : adj[a]) {
            int b = u.first;
            T w = u.second;

            if(dist[a] + w < dist[b]) {
                dist[b] = dist[a] + w;
                q.push({dist[b], b});
            }
        }
    }
}
```

Floyd-Warshall algorithm, all shortest paths with negatives.

floydwarshall.cpp

19 lines

```
// dist is adj matrix
void floydWarshall(int n, vector<vector<long long>>& dist) {
    for (int k = 0; k < n; ++k) {
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < n; ++j) {
                if (dist[i][k] < INF && dist[k][j] < INF) {
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
                }
            }
        }
    }

    for (int i = 0; i < n; ++i) {
        if (dist[i][i] < 0) {
            cout << "Graph contains a negative cycle reachable from vertex " << i <<
                 endl;
            return;
        }
    }
}
```

4.2 DSU

Add edges to graph, test if vertices are connected. Has path compression. This code is readable, but long.

Connectivity in $O(1)$, union in $O(N)$

dsu.cpp

25 lines

```
class DSU {
  private:
  vector<int> parents;
  vector<int> sizes;

  public:
  DSU(int size) : parents(size), sizes(size, 1) {
    for (int i = 0; i < size; i++) { parents[i] = i; }
  }

  int find(int x) { return parents[x] == x ? x : (parents[x] = find(parents[x])); }

  bool unite(int x, int y) {
    int x_root = find(x);
    int y_root = find(y);
    if (x_root == y_root) { return false; }

    if (sizes[x_root] < sizes[y_root]) { swap(x_root, y_root); }
    sizes[x_root] += sizes[y_root];
    parents[y_root] = x_root;
    return true;
  }
```

```
  bool connected(int x, int y) { return find(x) == find(y); }
};
```

4.3 DP

LIS in $O(N \log N)$

lisdp.cpp

35 lines

```
vector<int> lis(const vector<int> &a) {
    int n = a.size();
    if (n == 0) return {};

    vector<int> tails;
    vector<int> tails_indices;
    vector<int> parent(n, -1);

    for (int i = 0; i < n; i++) {
        auto it = lower_bound(tails.begin(), tails.end(), a[i]);
        int pos = distance(tails.begin(), it);

        if (it == tails.end()) {
            tails.push_back(a[i]);
            tails_indices.push_back(i);
        } else {
            *it = a[i];
            tails_indices[pos] = i;
        }

        if (pos > 0) {
            parent[i] = tails_indices[pos - 1];
        }
    }

    vector<int> lis;
    int curr = tails_indices.back();
    while (curr != -1) {
        lis.push_back(a[curr]);
        curr = parent[curr];
    }

    reverse(lis.begin(), lis.end());
    return lis;
}
```

Not really advanced, but bitset bashing is very good, can turn 10^9 operations into $\frac{10^9}{64}$ which is around $1.5 \cdot 10^7$.

Tree matching in $O(N)$ with greedy dfs, even though dp can be used.

treedp.cpp

10 lines

```
// start at root, needs adj & done arr, ans = 0.

void dfs(int pre, int cur) {
  for (int i : adj[cur]) {
    if (i != pre) {
      dfs(cur, i);
      if (!done[i] && !done[cur]) done[cur] = done[i] = 1, ans++;
    }
  }
}
```

Maybe i'll add a rerooting template.

4.4 Bashing Techniques

Too long, was removed. Files still exist though.

4.5 Range queries

Update element in $O(1)$, update range in $O(\sqrt{N})$ (currently not implemented tho), range query in $O(\sqrt{N})$.

sqrtblocking.cpp

27 lines

```
template <typename T>
struct Sqrt {
  int block_size;
  vector<T> nums;
  vector<T> blocks;
  T null_val;

  Sqrt(int& sqrtn, vector<T> &arr, T& null) : block_size(sqrtn), blocks(sqrtn, null)
      , null_val(null) {
    nums = arr;
    for (int i = 0; i < nums.size(); i++) { blocks[i / block_size] = op(blocks[i /
        block_size], nums[i]); }
  }

  void update(int& x, T& v) {
    blocks[x / block_size] = op_inv(blocks[x / block_size], nums[x]);
    nums[x] = v;
    blocks[x / block_size] = op(blocks[x / block_size], nums[x]);
  }

  T query(int& r) {
    T res = null_val;
    for (int i = 0; i < r / block_size; i++) {res = op(res, blocks[i]);}
    for (int i = (r / block_size) * block_size; i < r; i++) {res = op(res, nums[i])}
        ;
    return res;
  }

  T query(int& l, int& r) {return op_inv(query(r), query(l - 1))}
};
```

Update element in $O(\log N)$, range query in $O(\log N)$, range updates not implemented.

simplebit.cpp

33 lines

```cpp
template <typename T>
struct FenwickTree {
  int size;
  vector<T> bit;
  vector<T> arr;
  T null_val;

  BIT(int& size, T& null) : size(size), bit(size + 1), arr(size), null_val(null) {}

  void update(int& ind, T& val) {
    change(ind, null_val);
    change(ind, val);
  }

  void change(int& ind, T& val) {
    arr[ind] = op(arr[ind], val);
    ind++;
    for (; ind <= size; ind += ind & -ind) {bit[ind] = op(bit[ind], val);}
  }

  T query(int& ind) {
    ind++;
    T total = null_val;
    for (; ind > 0; ind -= ind & -ind) {total = op(total, bit[ind]);}
    return total;
  }

  T query(int& l, int& r) {
      if (l > r) return null_val;
      if (l == 0) return query(r);
      return query(r) - query(l - 1);
  }
};
```

Min/max of range in $O(1)$, $O(N)$ preprocessing

sparsetable.cpp

33 lines

```cpp
// op() is min or max only.
template <typename T>
struct SparseTable {
    int n;
    vector<vector<T>> st;

    SparseTable() {}

    SparseTable(vector<T>& arr) {
        n = arr.size();
        if (n == 0) return;

        int levels = _lg(n) + 1;
        st.assign(levels, vector<T>(n));

        for (int i = 0; i < n; i++) {
            st[0][i] = arr[i];
        }

        for (int j = 1; j < levels; j++) {
            for (int i = 0; i + (1 << j) <= n; i++) {
                st[j][i] = op(st[j - 1][i],
                              st[j - 1][i + (1 << (j - 1))]);
            }
        }
    }

    T query(int L, int R) {
        int len = R - L + 1;
        int j = _lg(len);
        return op(st[j][L], st[j][R - (1 << j) + 1]);
    }
};
```

Disjoint sparse table, any operation with query $O(1)$, $O(N \log N)$ preprocess.

disjointsptable.cpp

38 lines

```cpp
template<typename T>
struct DisjointSparseTable {
    int n, levels;
    vector<vector<T>> table;

    DisjointSparseTable(vector<T>& arr) {
        n = arr.size();
        levels = 32 - __builtin_clz(n);
        table.assign(levels, vector<T>(n));

        for (int i = 0; i < n; i++) table[0][i] = arr[i];

        for (int h = 1; h < levels; h++) {
            int len = (1 << h);
            for (int j = 0; j < n; j += len) {
                int mid = min(n, j + len / 2);
                // Suffixes from mid-1 down to j
                table[h][mid - 1] = arr[mid - 1];
                for (int k = mid - 2; k >= j; k--)
                    table[h][k] = op(arr[k], table[h][k + 1]);

                // Prefixes from mid up to j + len
                if (mid < n) {
                    table[h][mid] = arr[mid];
                    for (int k = mid + 1; k < min(n, j + len); k++)
                        table[h][k] = op(table[h][k - 1], arr[k]);
                }
            }
        }
    }

    T query(int& l, int& r) {
        if (l == r) return table[0][l];
```

```
        // Find the highest bit where l and r differ
        int h = 31 - __builtin_clz(l ^ r);
        return op(table[h][l], table[h][r]);
    }
};
```

4.5.1 Segment Trees

segtreebase.txt 35 lines

Since seg trees are very versatile, they have their own subsection.

```
template<typename T>
struct SegmentTree {
    int n;
    vector<T> tree;
    T ID = 0;

    SegmentTree(int n) : n(n), tree(4 * n, ID) {}

    void update(int pos, T new_val) {
        update(1, 0, n - 1, pos, new_val);
    }

    T query(int l, int r) {
        return query(1, 0, n - 1, l, r);
    }

private:
    void update(int v, int tl, int tr, int pos, T new_val) {
        if (tl == tr) tree[v] = new_val;
        else {
            int tm = (tl + tr) / 2;
            if (pos <= tm) update(2 * v, tl, tm, pos, new_val);
            else update(2 * v + 1, tm + 1, tr, pos, new_val);
            tree[v] = tree[2 * v] + tree[2 * v + 1];
        }
    }

    T query(int v, int tl, int tr, int l, int r) {
        if (l > r) return ID;
        if (l == tl && r == tr) return tree[v];
        int tm = (tl + tr) / 2;
        return query(2 * v, tl, tm, l, min(r, tm)) +
               query(2 * v + 1, tm + 1, tr, max(l, tm + 1), r);
    }
};
```

segtreelazy.cpp 56 lines

```
class LazySegTree {
private:
    int n;
    vector<long long> tree, lazy;

    void apply(int v, int tl, int tr, long long val) {
        tree[v] += val * (tr - tl + 1);
        lazy[v] += val;
    }

    void push(int v, int tl, int tr) {
        if (lazy[v] != 0) {
            int tm = (tl + tr) / 2;
            apply(2 * v, tl, tm, lazy[v]);
            apply(2 * v + 1, tm + 1, tr, lazy[v]);
            lazy[v] = 0;
        }
    }

    void update(int v, int tl, int tr, int l, int r, long long add) {
        if (l > r) return;
        if (l == tl && r == tr) {
            apply(v, tl, tr, add);
        } else {
            push(v, tl, tr);
            int tm = (tl + tr) / 2;
            update(2 * v, tl, tm, l, min(r, tm), add);
            update(2 * v + 1, tm + 1, tr, max(l, tm + 1), r, add);
            tree[v] = tree[2 * v] + tree[2 * v + 1];
        }
    }

    long long query(int v, int tl, int tr, int l, int r) {
        if (l > r) return 0;
        if (l == tl && r == tr) return tree[v];
        push(v, tl, tr);
        int tm = (tl + tr) / 2;
        return query(2 * v, tl, tm, l, min(r, tm)) +
               query(2 * v + 1, tm + 1, tr, max(l, tm + 1), r);
    }

public:
    LazySegTree(int _n) : n(_n) {
        tree.assign(4 * n, 0);
        lazy.assign(4 * n, 0);
    }

    // Public API: No more tl, tr!
    void update(int l, int r, long long val) {
        update(1, 0, n - 1, l, r, val);
    }

    long long query(int l, int r) {
        return query(1, 0, n - 1, l, r);
    }
};
```

segtreepersist.cpp 66 lines

```
class PersistentSegTree {
private:
    struct Node {
        int l, r;
```

```
        long long val; // In kth-smallest problems, this represents 'count'
        Node() : l(0), r(0), val(0) {}
    };

    vector<Node> tree;
    int n;

    int update(int prev_root, int tl, int tr, int pos, int val) {
        int node = tree.size();
        tree.push_back(tree[prev_root]); // Copy old node
        if (tl == tr) {
            tree[node].val += val; // Update the leaf (increment count)
            return node;
        }
        int tm = tl + (tr - tl) / 2;
        if (pos <= tm)
            tree[node].l = update(tree[prev_root].l, tl, tm, pos, val);
        else
            tree[node].r = update(tree[prev_root].r, tm + 1, tr, pos, val);

        tree[node].val = tree[tree[node].l].val + tree[tree[node].r].val;
        return node;
    }

    long long query(int v, int tl, int tr, int l, int r) {
        if (l > r || v == 0) return 0;
        if (l == tl && r == tr) return tree[v].val;
        int tm = tl + (tr - tl) / 2;
        return query(tree[v].l, tl, tm, l, min(r, tm)) +
               query(tree[v].r, tm + 1, tr, max(l, tm + 1), r);
    }

    int find_kth(int u, int v, int tl, int tr, int k) {
        if (tl == tr) return tl;
        long long left_count = tree[tree[v].l].val - tree[tree[u].l].val;
        int tm = tl + (tr - tl) / 2;
        if (left_count >= k)
            return find_kth(tree[u].l, tree[v].l, tl, tm, k);
        else
            return find_kth(tree[u].r, tree[v].r, tm + 1, tr, k - left_count);
    }

public:
    PersistentSegTree(int _n) : n(_n) {
        tree.reserve(2000000);
        tree.push_back(Node()); // Root 0 is the "null" node
    }

    int update(int prev_root, int pos, int val) {
        return update(prev_root, 0, n - 1, pos, val);
    }

    long long query(int root_version, int l, int r) {
        return query(root_version, 0, n - 1, l, r);
    }

    int find_kth(int root_l_minus_1, int root_r, int k) {
        return find_kth(root_l_minus_1, root_r, 0, n - 1, k);
    }

    int get_initial_root() { return 0; }
};
```

segtreesparse.cpp

62 lines

```
template<typename T>
class SparseSegTree {
private:
    struct Node {
        T val;
        int l, r; // Indices to children in the 'tree' vector
        Node() : val(0), l(-1), r(-1) {}
    };

    vector<Node> tree;
    long long L, R; // Range of the segment tree (e.g., 0 to 1e9)

    // Internal function to create a new node and return its index
    int newNode() {
        tree.emplace_back();
        return (int)tree.size() - 1;
    }

public:
    // Constructor defines the coordinate bounds
    SparseSegTree(long long min_coord, long long max_coord) : L(min_coord), R(
        max_coord) {
        tree.reserve(200000); // Pre-reserve to avoid frequent reallocations
        newNode(); // Create root at index 0
    }

    void update(long long pos, T delta) {
        update(0, L, R, pos, delta);
    }

    T query(long long ql, long long qr) {
        return query(0, L, R, ql, qr);
    }

private:
    void update(int v, long long tl, long long tr, long long pos, T delta) {
        if (tl == tr) {
            tree[v].val += delta;
            return;
        }
        long long tm = tl + (tr - tl) / 2;
        if (pos <= tm) {
            if (tree[v].l == -1) tree[v].l = newNode();
            update(tree[v].l, tl, tm, pos, delta);
        } else {
            if (tree[v].r == -1) tree[v].r = newNode();
            update(tree[v].r, tm + 1, tr, pos, delta);
```

```
        }

        T left_val = (tree[v].l != -1) ? tree[tree[v].l].val : 0;
        T right_val = (tree[v].r != -1) ? tree[tree[v].r].val : 0;
        tree[v].val = left_val + right_val;
    }

    T query(int v, long long tl, long long tr, long long ql, long long qr) {
        if (v == -1 || ql > tr || qr < tl) return 0;
        if (ql <= tl && tr <= qr) return tree[v].val;

        long long tm = tl + (tr - tl) / 2;
        return query(tree[v].l, tl, tm, ql, qr) +
               query(tree[v].r, tm + 1, tr, ql, qr);
    }
};
```

4.6 Trees

Still learning about uses.

eulertour.cpp

7 lines

```
void euler_tour(int at, int prev) {
  start[at] = timer++;
  for (int n : neighbors[at]) {
    if (n != prev) { euler_tour(n, at); }
  }
  end[at] = timer;
}
```

MST in $O(E \log E)$.

kruskal.cpp

9 lines

```
pair<int, int> edges[MAX_E];
// order edges by weight ascending
for (int i = 1; i <= V; i++) ms(i);
for (int i = 0; i < E; i++) {
    if (f(edges[i].first) != f(edges[i].second)) {
        // Add edges[i] to minumum spanning tree
        u(edges[i].first, edges[i].second);
    }
}
```

Longest path in dag in $O((V + E) \log E)$. If don't want the end part, just remove it. Topsort is good for only dags, but very useful for making sure nodes are all visited after ancestors.

kahn.cpp

33 lines

```
int kahn(int V, vector<vector<int>>& graph, vector<int>& deps) {
    vector<int> S;
    S.reserve(V);

    vector<int> dist(V + 1, 1);
    int max_dist = 1;
    int visited_count = 0;

    for (int i = 1; i <= V; i++) {
        if (deps[i] == 0) {
            S.push_back(i);
        }
    }

    while (!S.empty()) {
        int u = S.back();
        S.pop_back();
        visited_count++;

        for (int v : graph[u]) {
            if (dist[u] + 1 > dist[v]) {
                dist[v] = dist[u] + 1;
                if (dist[v] > max_dist) max_dist = dist[v];
            }

            if (--deps[v] == 0) {
                S.push_back(v);
            }
        }
    }

    return (visited_count == V) ? max_dist : -1;
}
```

LCA in $O(1)$, $O(N \log N)$ preprocessing.

lca.cpp

58 lines

```
class LCA {
private:
    int n, m;
    vector<int> first_visit, tour, depths, lg;
    vector<vector<int>> st;

    void dfs(const vector<vector<int>>& adj, int u, int p, int d) {
        first_visit[u] = tour.size();
        tour.push_back(u);
        depths.push_back(d);
        for (int v : adj[u]) {
            if (v != p) {
                dfs(adj, v, u, d + 1);
                tour.push_back(u);
                depths.push_back(d);
            }
        }
    }

public:
    LCA(const vector<vector<int>>& adj, int root = 0) {
        n = adj.size();
        first_visit.resize(n);
```

```
        tour.reserve(2 * n);
        depths.reserve(2 * n);

        dfs(adj, root, -1, 0);

        m = tour.size();
        int max_log = 31 - __builtin_clz(m) + 1;
        st.assign(max_log, vector<int>(m));
        lg.assign(m + 1, 0);

        for (int i = 2; i <= m; i++) lg[i] = lg[i / 2] + 1;
        for (int i = 0; i < m; i++) st[0][i] = i;

        for (int j = 1; j < max_log; j++) {
            for (int i = 0; i + (1 << j) <= m; i++) {
                int a = st[j - 1][i];
                int b = st[j - 1][i + (1 << (j - 1))];
                st[j][i] = (depths[a] < depths[b]) ? a : b;
            }
        }
    }

    int get_lca(int u, int v) {
        int L = first_visit[u], R = first_visit[v];
        if (L > R) swap(L, R);
        int j = lg[R - L + 1];
        int a = st[j][L];
        int b = st[j][R - (1 << j) + 1];
        return tour[(depths[a] < depths[b]) ? a : b];
    }

    int get_dist(int u, int v) {
        return depths[first_visit[u]] + depths[first_visit[v]] - 2 * depths[
            first_visit[get_lca(u, v)]];
    }
};
```

LCA in $O(\log N)$, easier to implement, binary jumping (find nth ancestor of node in $O(\log N)$ by pretty much making a sparse table of every 2^ith ancestor of each node)

binlift.cpp

42 lines

```
struct BinaryLifting {
    int n, max_log;
    vector<vector<int>> up;
    vector<int> tin, tout, depth;
    int timer;

    BinaryLifting(int n, int root, const vector<vector<int>>& adj) : n(n) {
        max_log = ceil(log2(n)) + 1;
        up.assign(n, vector<int>(max_log));
        tin.resize(n); tout.resize(n); depth.resize(n);
        timer = 0;
        dfs(root, root, 0, adj);
    }

    void dfs(int u, int p, int d, const vector<vector<int>>& adj) {
        tin[u] = ++timer;
        depth[u] = d;
        up[u][0] = p; // Parent of root is root
        for (int i = 1; i < max_log; i++) {
            up[u][i] = up[up[u][i-1]][i-1];
        }
        for (int v : adj[u]) {
            if (v != p) dfs(v, u, d + 1, adj);
        }
        tout[u] = ++timer;
    }

    bool is_ancestor(int u, int v) {
        return tin[u] <= tin[v] && tout[u] >= tout[v];
    }

    int get_lca(int u, int v) {
        if (is_ancestor(u, v)) return u;
        if (is_ancestor(v, u)) return v;
        for (int i = max_log - 1; i >= 0; i--) {
            if (!is_ancestor(up[u][i], v)) {
                u = up[u][i];
            }
        }
        return up[u][0];
    }
};
```

Also, to find nth ancestor of a node, just also use binary jumping.

4.7 Tree Structures

[TODO] hld, centroid decomposition, virtual tree, reconstruction tree

4.8 Advanced DP

A bunch of more advanced dp tricks/optimisations etc.

Also, I've heard of making dp more efficient by making into tree somehow.

4.9 Other graph theory

Find articulation points in $O(N + M)$

dfsartic.cpp

39 lines

```
int n; // number of nodes
vector<vector<int>> adj; // adjacency list of graph

vector<bool> visited;
vector<int> tin, low;
int timer;

// is_cutpoint means 'v' is an articulation point
```

```
void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children=0;
    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p!=-1)
                IS_CUTPOINT(v);
            ++children;
        }
    }
    if(p == -1 && children > 1)
        IS_CUTPOINT(v);
}

void find_cutpoints() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            dfs (i);
    }
}
```

Maximum matching in bipartite graph in $O(VE)$

kuhn.cpp

30 lines

```
struct Kuhns {
    int n, m;
    vector<vector<int>> adj;
    vector<int> match;
    vector<bool> vis;

    Kuhns(int n, int m) : n(n), m(m), adj(n), match(m, -1), vis(n) {}

    void add_edge(int u, int v) { adj[u].push_back(v); }

    bool dfs(int u) {
        vis[u] = true;
        for (int v : adj[u]) {
            if (match[v] < 0 || (!vis[match[v]] && dfs(match[v]))) {
                match[v] = u;
                return true;
            }
        }
        return false;
    }

    int max_matching() {
        int result = 0;
        for (int i = 0; i < n; i++) {
            fill(vis.begin(), vis.end(), false);
            if (dfs(i)) result++;
        }
        return result;
    }
};
```

Same thing but in $O(E\sqrt{V})$

hopcroftkarp.cpp

59 lines

```
struct HopcroftKarp {
    static const int INF = 1e9;
    vector<vector<int>> adj;
    vector<int> pairU, pairV, dist;
    int n, m;

    HopcroftKarp(int n, int m) : n(n), m(m), adj(n + 1), pairU(n + 1, 0), pairV(m +
        1, 0), dist(n + 1) {}

    void add_edge(int u, int v) { adj[u].push_back(v); }

    bool bfs() {
        queue<int> q;
        for (int u = 1; u <= n; u++) {
            if (pairU[u] == 0) {
                dist[u] = 0;
                q.push(u);
            } else dist[u] = INF;
        }
        dist[0] = INF;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            if (dist[u] < dist[0]) {
                for (int v : adj[u]) {
                    if (dist[pairV[v]] == INF) {
                        dist[pairV[v]] = dist[u] + 1;
                        q.push(pairV[v]);
                    }
                }
            }
        }
        return dist[0] != INF;
    }

    bool dfs(int u) {
        if (u != 0) {
            for (int v : adj[u]) {
                if (dist[pairV[v]] == dist[u] + 1) {
                    if (dfs(pairV[v])) {
                        pairV[v] = u;
                        pairU[u] = v;
```

```
                    return true;
                }
            }
        }
        dist[u] = INF;
        return false;
    }
    return true;
}

int max_matching() {
    int matching = 0;
    while (bfs()) {
        for (int u = 1; u <= n; u++)
            if (pairU[u] == 0 && dfs(u)) matching++;
    }
    return matching;
}
};
```

Stress Testing (5)

You should only EVER use this in one case, as a last resort (takes so long to implement and check), if the servers or school internet fails. But if you are cooked so badly that uve done everything you can, and went to the toilet etc, u can use this (why would you be).

tester.sh

45 lines

```
#!/bin/bash

# --- Configuration ---
TIME_LIMIT="2s"
NUM_TESTS=100
# ---------------------

echo "Compiling files..."
g++ sol.cpp -std=c++17 -O2 -o solution || exit 1
g++ -O3 gen.cpp -o gen || exit 1
g++ -O3 checker.cpp -o checker || exit 1

for ((i = 1; i <= NUM_TESTS; ++i)); do
    # 1. Generate test case
    ./gen $i > input.txt

    # 2. Run solution with timeout
    timeout $TIME_LIMIT ./solution < input.txt > out.txt
    exit_code=$?

    # 3. Handle Runtime Errors or Timeouts
    if [ $exit_code -eq 124 ]; then
        echo "TLE $i (> $TIME_LIMIT)"
        exit 1
    elif [ $exit_code -ne 0 ]; then
        echo "RE $i (Exit Code $exit_code)"
        echo "--- Input ---"
        cat input.txt
        exit 1
    fi

    # The checker reads input.txt and out.txt to verify validity
    if ! ./checker; then
        echo "WA $i"
        echo "--- Input ---"
        cat input.txt
        echo "--- Your Output ---"
        cat out.txt
        exit 1
    fi

    echo "AC $i"
done

echo "Successfully passed $NUM_TESTS tests!"
```

Here are some sample snippets for the checker and generator files:

gen.cpp

14 lines

```
#include <bits/stdc++.h>
using namespace std;

// Helper to get random numbers in range [l, r]
long long rng(long long l, long long r, mt19937& gen) {
    uniform_int_distribution<long long> dist(l, r);
    return dist(gen);
}

int main(int argc, char* argv[]) {
    // Seed is from bash file, u don't have to use tho
    int seed = atoi(argv[1]);
    mt19937 gen(seed);
}
```

checker.cpp

19 lines

```
#include <bits/stdc++.h>
using namespace std;

/**
 * EXIT CODES:
 * 0: AC (Accepted)
 * 1: WA (Wrong Answer / Logic Error)
 */

int main() {
    ifstream fin("input.txt");    // The problem input
    ifstream fout("out.txt");     // Your solution's output

    // Use fin and fout to get input, make sure format is correct first.
    // If anything goes wrong, return 1.

    // If we reached here, everything is correct
    return 0;
}
```

Miscellaneous (6)

6.1 Binary Search Variants

Ternary Search: Used to find min/max of unimodal function

ternsearch.cpp

14 lines

```
double ternary_search(double l, double r) {
    //set error limit
    double eps = 1e-9;
    while (r - l > eps) {
        double m1 = l + (r - l) / 3;
        double m2 = r - (r - l) / 3;
        double f1 = f(m1);
        double f2 = f(m2);
        if (f1 < f2) l = m1;
        else r = m2;
    }
    //return the maximum of f(x) in [l, r]
    return f(l);
}
```

Binary Search on psuedo-derivative (forward differences): Same application as ternary search.

bsearchdx.cpp

10 lines

```
double low = 0, high = 1e9;
double h = 1e-9; // Your delta

for(int i = 0; i < 100; i++) {
    double mid = low + (high - low) / 2.0;

    if (f(mid + h) > f(mid)) high = mid;
    else low = mid;
}
double min_x = low;
```

6.2 Minimum-storing data structures

Currently don't have initialisers yet.

Just a queue with more constant factor, but can get min/max element in $O(1)$

minqueue.cpp

23 lines

```
template <typename T>
struct MinQueue {
    deque<T> q;       // Stores all elements currently in the queue
    deque<T> min_q;   // Stores elements in non-decreasing order

    void push(T& val) {
        q.push_back(val);
        while (!min_q.empty() and min_q.back() > val) {
```

```
            min_q.pop_back();
        }
        min_q.push_back(val);
    }

    void pop() {
        if (q.empty()) return;
        if (q.front() == min_q.front()) min_q.pop_front();
        q.pop_front();
    }

    T get_min() return min_q.front();
    bool empty() return q.empty();
    int size() return q.size();
};
```

Just a stack with more constant factor, can get min/max in $O(1)$

minstack.cpp

19 lines

```
template <typename T>
struct MinStack {
    vector<pair<T, T>> s;

    void push(T& val) {
        if (s.empty()) {
            s.push_back({val, val});
        } else {
            T current_min = min(val, s.back().second);
            s.push_back({val, current_min});
        }
    }

    void pop() if (!s.empty()) s.pop_back();
    T top() return s.back().first;
    T get_min() return s.back().second;
    bool empty() return s.empty();
    int size() return s.size();
};
```

6.3 Order Statistic Tree

orderedset.cpp

8 lines

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

typedef tree<int, null_type, less<int>, rb_tree_tag,
    tree_order_statistics_node_update> ordered_set;

// find_by_order(k): Iterator to set[k]
// order_of_key(n): set.index(n), set.index(n - smth)
```

orderedmultiset.cpp

44 lines

```
template <typename T>
class ordered_multiset {
    // Pair: (value, unique_id)
    typedef pair<T, long long> TPair;
    struct cmp {
        bool operator()(const TPair &a, const TPair &b) const {
            if (a.first != b.first) return a.first < b.first;
            return a.second < b.second; // tie-break by unique ID
        }
    };

    typedef tree<
        TPair,
        null_type,
        cmp,
        rb_tree_tag,
        tree_order_statistics_node_update
    > ost_type;

    ost_type ost;
    long long uid = 0; // automatic ID counter

    void insert(const T &value) ost.insert({value, uid++});

    // Erase one occurrence
    bool erase(const T &value) {
        auto it = ost.lower_bound({value, LLONG_MIN});
        if (it != ost.end() && it->first == value) {
            ost.erase(it);
            return true;
        }
        return false;
    }

    size_t order_of_key(const T &value) const return ost.order_of_key({value,
        LLONG_MIN});

    optional<T> find_by_order(size_t k) const {
        if (k >= ost.size()) return nullopt;
        return ost.find_by_order(k)->first;
    }

    size_t size() const return ost.size();
    bool empty() const return ost.empty();
};
```

orderedmap.cpp

8 lines

```
template <typename Key, typename Value>
using ordered_map = tree<
    Key,
    Value,
    less<Key>,
    rb_tree_tag,
```

```
    tree_order_statistics_node_update
>;
```

6.4 Mathematics

A bunch of geometry implementations and combi/nt.

lcmgcd.cpp

3 lines

```
int gcd(int a, int b) return b == 0 ? a : gcd(b, a % b);

int lcm(int a, int b) return a*b / gcd(a, b);
```

binexp.cpp

11 lines

```
ll exp(ll x, ll n, ll m) {
  assert(n >= 0);
  x %= m;
  ll res = 1;
  while (n > 0) {
    if (n % 2 == 1) { res = res * x % m; }
    x = x * x % m;
    n /= 2;
  }
  return res;
}
```

modularint.cpp

28 lines

```
struct mi {
  int v;
  explicit operator int() const { return v; }
  mi() { v = 0; }
  mi(long long _v) : v(_v % MOD) { v += (v < 0) * MOD; }
};

mi &operator+=(mi &a, mi b) {
  if ((a.v += b.v) >= MOD) a.v -= MOD;
  return a;
}
mi &operator-=(mi &a, mi b) {
  if ((a.v -= b.v) < 0) a.v += MOD;
  return a;
}
mi operator+(mi a, mi b) { return a += b; }
mi operator-(mi a, mi b) { return a -= b; }
mi operator*(mi a, mi b) { return mi((long long)a.v * b.v); }
mi &operator*=(mi &a, mi b) { return a = a * b; }
mi pow(mi a, long long p) {
  assert(p >= 0);
  return p == 0 ? 1 : pow(a * a, p / 2) * (p & 1 ? a : 1);
}
mi inv(mi a) {
  assert(a.v != 0);
  return pow(a, MOD - 2);
}
mi operator/(mi a, mi b) { return a * inv(b); }
```

If u don't want to fully write out the mod int class, but need division.

modinverse.cpp

16 lines

```
// Only works if M is prime, uses lls and bin exp

long long power(long long base, long long exp) {
    long long res = 1;
    base %= MOD; // Use your M here
    while (exp > 0) {
        if (exp % 2 == 1) res = (res * base) % MOD;
        base = (base * base) % MOD;
        exp /= 2;
    }
    return res;
}

long long modInverse(long long n, long long MOD = 1000000007) {
    return power(n, M - 2);
}
```

rectinterset.cpp

8 lines

```
struct Rect {int x1, y1, x2, y2;};

int intersect(Rect p, Rect q) {
  // Calculate overlap in x and y directions
  int xOverlap = max(0, min(p.x2, q.x2) - max(p.x1, q.x1));
  int yOverlap = max(0, min(p.y2, q.y2) - max(p.y1, q.y1));
  return xOverlap * yOverlap;  // Area of intersection
}
```

polyarea.cpp

11 lines

```
struct point {double x, y;};

double area(const vector<point>& fig) {
    double res = 0;
    for (unsigned i = 0; i < fig.size(); i++) {
        point p = i ? fig[i - 1] : fig.back();
        point q = fig[i];
        res += (p.x - q.x) * (p.y + q.y);
    }
    return fabs(res) / 2;
}
```

A bunch of tricks for vectors & points.

point.cpp

27 lines

```
struct Point {
    ll x, y;

    Point operator+(const Point& o) const { return {x + o.x, y + o.y}; }
    Point operator-(const Point& o) const { return {x - o.x, y - o.y}; }
    bool operator==(const Point& o) const { return x == o.x && y == o.y; }
```

```
    ll dot(const Point& o) const { return x * o.x + y * o.y; }
    ll cross(const Point& o) const { return x * o.y - y * o.x; }

    // Use for: Comparing distances without using sqrt()
    ll distSq(const Point& o) const {
        return (x - o.x) * (x - o.x) + (y - o.y) * (y - o.y);
    }

    // 1 (Left), -1 (Right), 0 (Collinear)
    int cross_prod_sign(Point b, Point c) const {
        ll val = (b - *this).cross(c - *this);
        if (val == 0) return 0;
        return (val > 0) ? 1 : -1;
    }

    // Lies on AB
    bool onSegment(Point a, Point b) const {
        return cross_prod_sign(a, b) == 0 && (a - *this).dot(b - *this) <= 0;
    }
};
```

6.4.1 Sweep Line

Pretty trivial, just do at endpoints of each line. Also remember radial sweeps, might need to be optimised a bunch tho.

6.5 Bits

Too lazy for explanation, here is reference for bit operations. Please note that all builtin functions have a long long version, add 'll' to end.

bits.cpp

8 lines

```
x | y; // OR
x & y; // AND
x ^ y; // XOR
x << y; // x * 2 pow y
x >> y; // floor(x / 2 pow y)
__builtinpopcount(x); // Ones in x in base 2
__builtinctz(x); // Trailing 0s in x in base 2
__builtinclz(x); // Leading 0s in x in base 2
```

Bitsets: each operation costs at most $O(\frac{size}{32})$, some can be $O(\frac{size}{64})$. Usually used to represent sets, meaning you can merge, see common elements, get count of set bits, toggle bits, set bits etc very quickly compared to other ds.

bitset.cpp

10 lines

```
// Initialiser can be a number, string of 0s and 1s. Automatically set to 0000...
// Size MUST be defined at compile-time. Just define as max possible value
bitset<size> x(initialiser);

// Use normal operations such as &, |, ^, ~, <<, >> to find stuff like nth element
    etc
// Can be indexed, can use .size()
x.flip(); // Toggle
x.set(); // Set to 11111...
x.reset(); // Set to 0000...
x._Find_first(); // First set bit index
```

Attempt to make custom bitset class which turns $O(\frac{N}{64})$ into $O(\frac{N}{256})$ instead. Untested on orac tho.

fastbitset.cpp

64 lines

```
// Needs pragmas
typedef uint64_t v256 __attribute__ ((vector_size (32), aligned (32)));

struct FastBitset {
    vector<v256> blocks;
    size_t n_bits;

    FastBitset(size_t n) : n_bits(n) {
        blocks.resize((n + 255) / 256, (v256){0});
    }

    // O(N/256)
    void operator&=(const SuperBitset& other) {
        for (size_t i = 0; i < blocks.size(); ++i) blocks[i] &= other.blocks[i];
    }
    void operator|=(const SuperBitset& other) {
        for (size_t i = 0; i < blocks.size(); ++i) blocks[i] |= other.blocks[i];
    }
    void operator^=(const SuperBitset& other) {
        for (size_t i = 0; i < blocks.size(); ++i) blocks[i] ^= other.blocks[i];
    }
    void flip() {
        v256 mask = (v256)_mm256_set1_epi64x(-1LL);
        for (auto& b : blocks) b ^= mask;
    }

    // O(N/64)
    size_t count() const {
        size_t cnt = 0;
        const uint64_t* p = (const uint64_t*)blocks.data();
        for (size_t i = 0; i < blocks.size() * 4; ++i) cnt += __builtin_popcountll(p
            [i]);
        return cnt;
    }

    bool any() const {
        for (const auto& b : blocks) {
            if (!_mm256_testz_si256(b, b)) return true;
        }
        return false;
    }

    // O(N/64)
    void operator<<=(size_t shift) {
        if (shift == 0) return;
        size_t word_shift = shift >> 6;
        size_t bit_shift = shift & 63;
```

```
        uint64_t* p = (uint64_t*)blocks.data();
        int n = blocks.size() * 4;

        if (bit_shift == 0) {
            for (int i = n - 1; i >= (int)word_shift; --i) p[i] = p[i - word_shift];
        } else {
            for (int i = n - 1; i >= (int)word_shift; --i) {
                uint64_t high = p[i - word_shift] << bit_shift;
                uint64_t low = (i - word_shift > 0) ? (p[i - word_shift - 1] >> (64
                    - bit_shift)) : 0;
                p[i] = high | low;
            }
        }
        for (size_t i = 0; i < word_shift; ++i) p[i] = 0;
    }

    void set(size_t i) { ((uint64_t*)blocks.data())[i >> 6] |= (1ULL << (i & 63)); }
    bool test(size_t i) const { return (((uint64_t*)blocks.data())[i >> 6] >> (i &
        63)) & 1; }
};
```

6.6 Other

cpptricks.cpp

15 lines

```
// Cool trick: if a is b, cout << " " else, cout "\n". Useful for graphs
cout << " \n"[a == b];

// Custom literals!
long long operator "" _km(unsigned long long literal) {return literal * 1000};
8_km // returns 8000

// If x is != 0, returns 1, else 0
!!x

// Initialisers in ifs, works with whiles
if(int x = f(); is_good(x)) //

// Commas in fors, also works with ifs, whiles etc
for(int a = 0, int b = 0; a != b; a++) //
```

Merge sets in $O(m \log n)$, improvement over $O((m+n)\log n)$ (i think). Note that for gnu pbds, use a.swap(b) instead. Also, copying ds into set neatly.

sets.cpp

32 lines

```
// Merge sets in O(M log N), instead of O((M + N) log N)
if (a.size() < b.size()) swap(a, b);
for (int x : b) a.insert(x);

// Nice way for set initialisation
set<int> S(vec.begin(), vec.end());

// Check if element exists
binary_search(vec.begin(), vec.end(), key);

// Extended binary search, p(x) = true, find where p(x) = false first.
partition_point(vec.begin(), vec.end(), p);

// Pointer tricks
iter - vec.begin(); // Distance to start (index)
vec.end() - 1; // Distance to end, also last element

// Lambdas
[capture_list](params) -> T {
    // capture is either [], [=], [&]
    // params are like normal c++ function params
    // return type (T) usually automatically detected.
    // Code
};

// std::views (ONLY C++20 and above)
for(T& elem : container
    | views::filter([](T val) -> bool {})
    | views::transform([](T val) -> T {})
) {
    //...
}
```

Not included, but [SIMD vectorisation], for speeding up brute force algos even more.

6.7 Practice

With this book, and maybe some other stuff not really in AIO (i.e. bitwise operators), you can probably easily solve every single problem from the Atcoder Weekday Contests (AWC). Very serious. Start solving them, 5 questions a day, very easy and good, try do them live.

Then, Atcoder Beginner Contests (ABC) is very similar to AIO level (maybe a bit harder), so good for practice. Maybe not virtual, because time limit is quite tight, but can help with solving speed. For AIIO/FARIO Practice, do Virtual ARC– and ARC normal contests (ARC++ can be quite hard).

Next, go to orac website by amt, and sequentially solve all problems from 1 star to 5 stars (also very good for practice - hard problems have no editorials and usually no online solutions). Try get on the leaderboard as well.

Install the orac userscript from github using a userscript manager such as tampermonkey to enhance your experience (lots of good features, more updates coming as of 2026 April).

Also, cf catalog can be good for learning topics.

However, don't practice when you don't feel like it (unless you get super lazy), because usually it won't be productive and you can do better things with your time. The best way to get better is just grind problems, and don't lie to yourself.

Check cppreference for cool features of c++, but keep in mind that AIO is apparently still using C++11, but I'm pretty sure its at C++17. C++20, 23, 26 features are very cool though.

6.8 Environment

If ur on windows, you should use wsl so that you can keep ¡bits/stdc++.h¿, while being able to use sanitisation flags. Also, using wsl gives you more accurate timing (explained in next paragraph)

I use vs code, good integration with wsl. Also, I like to use many extensions, such as CPH, DiffChecker, cppReference inside vs code, note taker. (CPH has lots of extensions, in wsl it has more accurate timing)

6.8.1 Physical & Mental Environment

Make sure you are not distracted, and you are not thinking hard on something else. Also make sure it's comfortable.

When you get a WA or smth like that just go eh too bad and continue locking in (virtual contests of AWC/ABC or live on atcoder are very good for practicing mindset).

6.9 Topic Searching

Here's a big list of topics/patterns to search for.

topics.txt

17 lines

```
Binary Search on ans
Binary Search
Greedy
Monotone Stack
Prefix Sums
Sliding Window
2 Pointers
DP
Interval
Scheduling
Brute Force
Difference Array
Interval Merging with greedy sorting
Greedy sorting and binary search
Shortest path
Bashing with bm & matrices (or fft)
... i need to add more, maybe as i solve problems
```